

Pipeline MLOps

Metodologías de desarrollo y
despliegue de aplicaciones para
ciencia de datos e IA

RICHARD MAKOV'S SELEZNOVA

1. INTRODUCCIÓN	3
2. ARQUITECTURA DEL SISTEMA	4
3. DECISIONES TÉCNICAS	4
4. METODOLOGÍA ÁGIL	5
5. CONCLUSIONES	7

1. INTRODUCCIÓN

Este trabajo presenta el diseño y la implementación de un pipeline MLOps construido sobre un caso de uso de clasificación supervisada, donde el objetivo ha sido demostrar el conjunto de prácticas, herramientas y decisiones de ingeniería que permiten que un proyecto de ciencia de datos pase a un servicio público y accesible, para ello se ha elegido el dataset Iris junto con un clasificador Random Forest.

El sistema desarrollado está formado por varios componentes que trabajan de forma integrada y automática. Por un lado, se encuentra el código de entrenamiento e inferencia en Python, utilizando scikit-learn para el modelo y FastAPI junto con Mangum para las predicciones a través de una API REST, además, se han implementado pruebas con pytest, con una cobertura del 85 % del código y se ejecutan de manera automática con cualquier cambio. Luego, se ha creado una imagen Docker basada en la imagen oficial de AWS Lambda para Python, permitiendo que funcione tanto en entornos locales como en la nube sin modificaciones, también con una integración y despliegue continuo definidos en GitHub Actions, organizados en dos procesos: validación del código y su despliegue. Por último, toda la infraestructura en AWS ha sido definida mediante Terraform, incluyendo un repositorio ECR para almacenar las imágenes, una función Lambda contenedorizada y una Function URL pública accesible mediante HTTPS.

El resultado final es una API en producción en la que cada modificación del código en la rama principal llama de forma automática la ejecución de las pruebas, el reentrenamiento del modelo, la construcción de una nueva imagen, su publicación en el registro privado y la actualización del servicio en producción. Durante el proceso fue necesario tomar decisiones técnicas que se desviaron del planteamiento inicial como la migración desde AWS App Runner hacia AWS Lambda debido a la decisión de Amazon de cerrar App Runner a nuevos clientes.



Aviso

Starting April 30, 2026, AWS App Runner is no longer accepting new customers. Your existing App Runner services will remain operational and accessible. AWS continues to invest in security and availability, but we do not plan to introduce new features. For deploying and running containerized applications, we recommend [Modo exprés de Amazon ECS](#). To learn more, see [Modo exprés de Amazon ECS](#) and the [guía de migración](#).

Informática

AWS App Runner

Cree y ejecute aplicaciones web de producción a escala

AWS App Runner es un servicio completamente administrado que facilita a los desarrolladores la implementación desde código fuente o imagen de contenedor directamente en una aplicación web escalable y segura.

Costos informáticos (EE. UU.)

2. ARQUITECTURA DEL SISTEMA

La arquitectura sigue un pipeline MLOps en torno al repositorio de GitHub, cuando un desarrollador realiza un push de cambios sobre la rama principal, GitHub dispara dos workflows en paralelo, el primero verifica la calidad del código mediante la ejecución de la suite de tests y la comprobación de que la cobertura supera el umbral mínimo del setenta por ciento exigido, mientras que el segundo asume que la calidad ya está validada y se encarga de construir la imagen Docker, etiquetarla, subirla al registro ECR de AWS y ordenar a la función Lambda que adopte la nueva imagen.

Una vez la imagen está en ECR, AWS Lambda se actualiza y las peticiones que lleguen a la Function URL pasarán a ser procesadas por el contenedor recién desplegado. La Function URL es una característica de Lambda que expone una URL HTTPS pública sin necesidad de configurar un API Gateway ni un balanceador. Dentro del contenedor, la aplicación FastAPI recibe la petición a través del adaptador Mangum que se encarga de traducir el formato de evento Lambda al protocolo ASGI que FastAPI entiende, ejecuta el endpoint correspondiente y devuelve la respuesta.

La separación lógica del código sigue una estructura con la zona del modelo ubicada en el módulo de entrenamiento, se encarga de cargar el dataset, dividirlo en conjuntos de entrenamiento y prueba, ajustar el clasificador y persistir en disco mediante joblib. La zona de servicio (separada en otro módulo) define la aplicación FastAPI con sus modelos Pydantic de entrada y salida, sus endpoints de salud y predicción, y el handler que la conecta con Lambda. La zona de pruebas (aislada) contiene tests unitarios sobre las funciones del entrenamiento y tests de integración sobre la API utilizando el TestClient de FastAPI, y por último la zona de infraestructura, dos partes: el Dockerfile y los workflows, y los archivos Terraform.

3. DECISIONES TÉCNICAS

A lo largo del desarrollo se tomaron varias decisiones, tanto por las que se eligieron como por las alternativas que se descartaron. La primera y más visible fue la elección del dataset y del modelo. Se optó por Iris y por un Random Forest lo que permite que la atención se concentre en el pipeline, ya que se ha asumido que la accuracy del 100% no es indicativa de la calidad de la modelización.

En cuanto a la API, la elección de FastAPI en vez de Flask es por tres razones: es la validación automática de entradas a través de Pydantic que elimina líneas de código de validación manual y produce mensajes de error claros y consistentes, luego la documentación interactiva que se genera de forma automática a partir de las anotaciones

de tipo, accesible en la ruta /docs, y la tercera es que FastAPI se ha convertido en el estándar para servir modelos en MLOps.

La intención era utilizar AWS App Runner por que se le entrega una imagen y devuelve una URL pública (sin la complejidad de configurar VPCs), balanceadores ni grupos de seguridad, sin embargo, durante la fase de despliegue AWS había anunciado el cierre de App Runner para nuevos clientes, entonces tras evaluar las alternativas se descartó ECS Fargate clásico por su exceso de configuración para un caso de uso tan acotado y se descartó ECS Express, y finalmente se eligió AWS Lambda con imagen de contenedor y Function URL, porque el coste es nulo ya que su modelo de pago es por invocación y por su free tier.

Al implementar Lambda se ha adaptado el código, ya que no ejecuta uvicorn como un servidor web sino que invoca una función handler con un evento estructurado, por lo que fue necesario incorporar Mangum (librería que actúa como puente entre el modelo de invocación de Lambda y la aplicación ASGI de FastAPI).

Otra decisión fue la de gestionar la infraestructura mediante Terraform debido a: la reproducibilidad: cualquier persona con las credenciales adecuadas puede ejecutar terraform apply y obtener una infraestructura idéntica, la trazabilidad: cada cambio en la infraestructura queda en el historial de Git como cualquier otro cambio de código, y la destrucción: con un terraform destroy puedes eliminar todos los recursos creados.

Finalmente, sobre la separación entre CI y CD se decidió configurar dos workflows distintos en lugar de un único pipeline porque el workflow de integración se ejecuta en cada push y en cada pull request, mientras que el de despliegue solo se dispara en la rama principal, esto permite que al abrir una pull request y recibir feedback sobre la calidad del código sin que se construya y publique una imagen.

4. METODOLOGÍA ÁGIL

La gestión del proyecto se ha realizado con Scrum adaptado a un desarrollador y plazo comprimido, se ha mantenido la estructura iterativa dividiendo el trabajo en tres ciclos cortos, cada uno con su propio objetivo, su backlog específico y su revisión al final.

El primer sprint tuvo como objetivo construir la base local del sistema y abarcó la creación del repositorio, la estructura de directorios, el script de entrenamiento del modelo, la API con FastAPI y la suite de tests, luego el backlog incluía historias relacionadas con la capacidad de entrenar el modelo desde un solo comando, la posibilidad de probar predicciones desde un navegador local, y que cualquier nueva funcionalidad tuviera su test correspondiente. Al cierre del sprint el sistema funcionaba en local, con la API respondiendo en localhost y los tests cubriendo más del setenta por ciento del código.

El segundo sprint se centró en la contenedorización y la integración continua, se construyó el Dockerfile inicial, se configuró el workflow de CI con ejecución de tests y comprobación de cobertura, y se preparó el repositorio ECR mediante Terraform, también fue importante garantizar que la imagen funcionase tanto en local como en cualquier máquina de un colaborador, lo que llevó a tomar la decisión de no versionar el modelo entrenado en Git sino regenerarlo durante el pipeline. Al cierre del sprint cualquier push al repositorio disparaba la ejecución de pruebas automatizadas y se disponía de un registro de imágenes en AWS.

El tercer sprint cerró el ciclo añadiendo el despliegue continuo, la API pública en internet y la documentación, se produjo la decisión de migrar de App Runner a Lambda y la refactorización del Dockerfile y del módulo de la API, donde también se elaboraron el README, la memoria, el sprint se cerró con el pipeline funcionando de extremo a extremo y la API accesible desde cualquier navegador.

Al final del tercer sprint se llevó a cabo una retrospectiva personal sobre lo que había funcionado bien, lo que se podría haber hecho mejor y los aprendizajes para futuros proyectos, entre ellos la decisión temprana de separar el código en zonas claras y usar funciones reutilizables facilitó los tests, entre los puntos a mejorar fue la dependencia inicial sobre App Runner, otro punto a mejorar fue la gestión de permisos IAM en AWS, donde varios despliegues fallaron porque el usuario no tenía políticas suficientes; en proyectos futuros preparar de antemano una política consolidada con todos los permisos previstos. Como aprendizaje un pipeline MLOps no es una herramienta que se monte de una vez sino un conjunto de piezas cuyas dependencias externas pueden cambiar y que deben ser evaluadas.

El tablero del proyecto en GitHub Projects contiene esta estructura mediante un Kanban con las columnas habituales de Todo, In Progress y Done, donde cada historia de usuario ha ido moviéndose conforme avanzaba el trabajo.

5. CONCLUSIONES

En el trabajo se ha diseñado, implementado y desplegado un pipeline MLOps completo aplicando metodologías ágiles y herramientas de la industria, el desarrollo ha permitido consolidar una serie de aprendizajes.

El primero es que la complejidad de un proyecto MLOps reside en la integración de cada una de las piezas individuales, cada herramienta tiene su propio lenguaje, sus propias convenciones y sus propios fallos sutiles, y la mayor parte del esfuerzo se va en lograr que todas hablen entre sí de manera fiable.

El segundo es la importancia de la infraestructura como código, declarar la infraestructura en Terraform aporta beneficios ya que cualquier ajuste de configuración se realiza editando un archivo y aplicando, en lugar de recordar dónde estaba cada apartado en el panel de AWS, y la posibilidad de destruir y recrear el entorno completo.

El tercer aprendizaje es la potencia del modelo de Continuous Deployment cuando está bien configurado, una vez los workflows estaban bien configurados es bastante cómodo que al hacer push de unos cambios al repositorio luego verlo en producción después de unos minutos, ya que permite hacer cambios pequeños y frecuentes.

De cara a posibles mejoras del proyecto, hay varias líneas que se han quedado fuera por el tiempo pero que serían los siguientes pasos: la autenticación de GitHub Actions debería migrar desde claves de larga duración hacia OIDC con roles asumibles porque elimina la el almacenar secrets keys en GitHub, la imagen Docker podría reducirse mediante un build multi-etapa, los permisos IAM podrían restringirse pasando a ARNs concretos, y se podría añadir una capa de observabilidad con logging estructurado, métricas y trazas. Cualquiera de estas mejoras podría abordarse en un sprint adicional sin afectar al resto del sistema.